

AI-Native Self-Healing O-RAN Network

Using a Digital Twin for Open-Fronthaul S-plane Recovery

From passive timing telemetry to governed recovery before the ~2 s failure window

CPU-only emulation

Labelled dataset

Hardware-ready path

100 ns

S-plane time-error budget

~2 s

attack-to-failure window

0.547 s

measured governed-loop
MTTR

Author: Jaswanth | Status: reproducible simulator + benchmark

The main topic becomes one governed loop

Each word maps to a concrete software responsibility, not a slogan.



Concrete thesis: detect a sync anomaly, separate benign timing faults from malicious attacks, verify recovery actions in a twin, then commit the safest action before the base station fails.

Timing is a control-plane dependency, not background plumbing

PTP + SyncE keep the O-DU and O-RU aligned tightly enough for radio operation.

PTP + SyncE

Open-fronthaul synchronization stack

~100 ns

target time-error class

~2 s

reported crash window
under spoofing

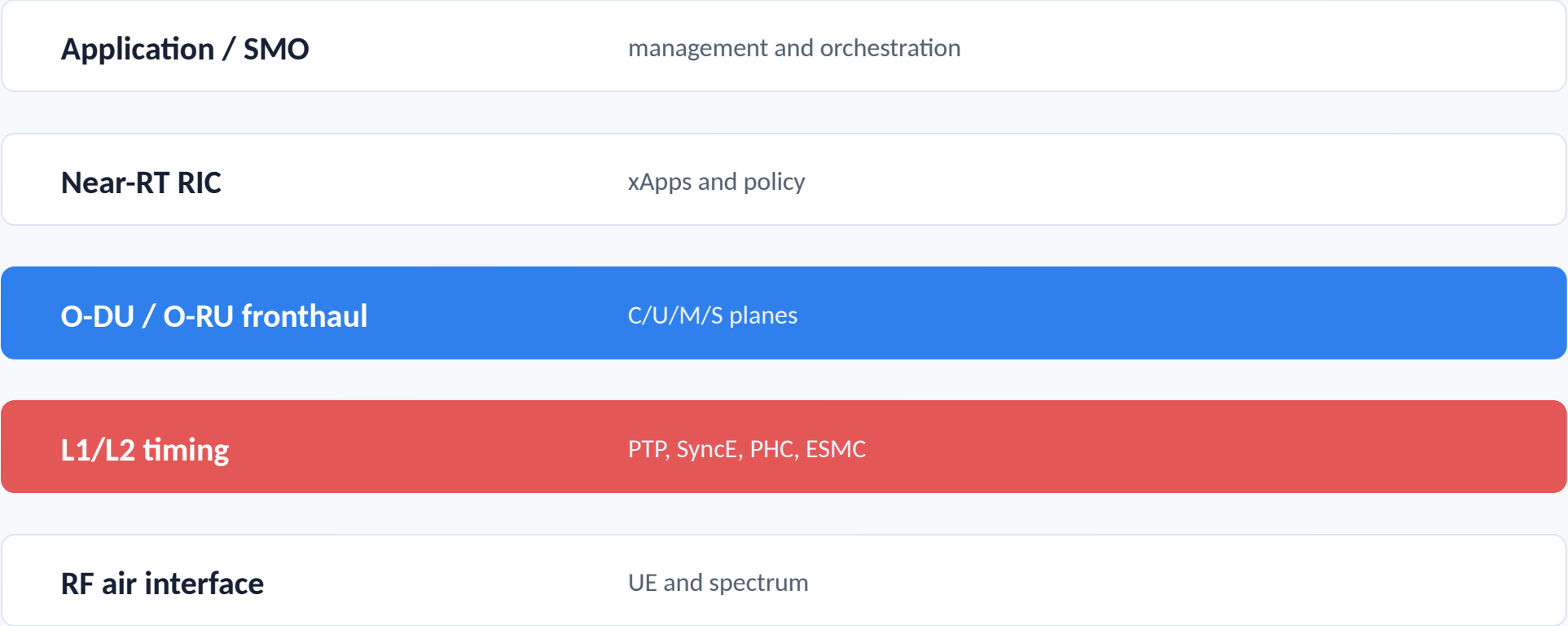
O-DU ↔ O-RU

timing domain, not Internet
routing

- PTP distributes time; SyncE helps frequency stability.
- A forged or replayed master can move the slave clock outside budget.
- Recovery must be selected before the failure window, not after outage evidence arrives.

The project sits in the fronthaul S-plane

It is not a routing problem and it is not the radio air interface.



Highlighted scope: Open-Fronthaul S-plane

Prior work proves the threat; response is the gap

The gap is action selection and fault-vs-attack separation.

Work	What it does	What remains open
TIMESAFE, ACM ToPS 2025	Shows fronthaul timing attacks and reports a 97.5% detector	Stops at detection; no recovery action; no H0/H1 split
OpenTwin / DTN work	Uses twins for network optimization and data/energy studies	Not focused on S-plane attack recovery
xApp conflict mitigation	Manages conflicting RIC control actions	Different layer; not PTP/SyncE timing
This project	Discriminate H0/H1, verify actions in a twin, commit before failure	Emulated today; hardware validation is next

The formal problem is response under uncertainty

Contribution: the missing response step plus fault/attack split and a released dataset.

Input

- offset, mean path delay, PDV
- PTP sequence/message regularity
- SyncE QL, GNSS and holdover flags

Decision

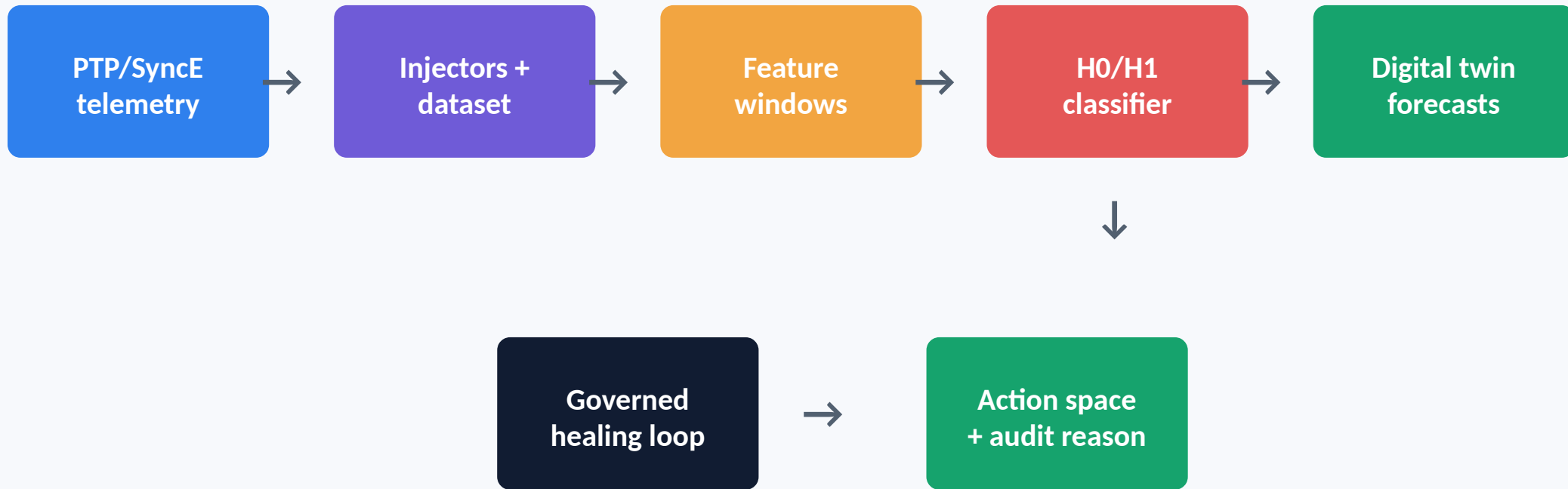
- detect anomaly
- classify H0 benign fault vs H1 attack
- forecast each action in the twin
- commit only if it beats safe default

Main-topic mapping: O-RAN = fronthaul; AI-native = learned discrimination; self-healing = autonomous action; digital twin = governed verification.

Honest scope: integration + experimental validation + 2,508-window released dataset; not a new fundamental algorithm.

System architecture turns telemetry into an auditable action

No ordering was present, so this diagram was generated from the implemented modules.



The simulator gives a repeatable S-plane world

fronthaul_sim/simulator.py

01

**A clock-servo model
produces PTP/SyncE
telemetry on a laptop.**

fronthaul_sim/simulator.py

P1 simulator

Key technical terms

- PTP offset
- mean path delay
- PDV
- frequency error
- SyncE QL

What it does

- Disciplines a slave clock
- Supports GNSS reference and holdover
- Keeps healthy traces within budget

Injectors create labelled benign faults and attacks

faults/injectors.py

02

**Scenario injectors
produce timestamped H0,
H1, and healthy labels.**

faults/injectors.py

P2 labels

Key technical terms

- GNSS loss
- PDV burst
- SyncE degrade
- PTP spoof
- replay

What it does

- Covers three H0 fault modes
- Covers two H1 attack modes
- Hardening adds overlap and mimicry

Feature extraction turns raw telemetry into 0.4 s decisions

telemetry/features.py

03

**Sliding windows
summarize timing
behavior for classification.**

telemetry/features.py

10 features

Key technical terms

- 0.4 s window
- 0.2 s step
- offset stats
- sequence regressions
- majority label

What it does

- Builds ten model inputs
- Preserves scenario/run identity
- Outputs anomaly labels per window

The dataset builder makes the experiment reproducible

dataset/build.py

04

One script assembles telemetry, labelled windows, and the datasheet.

dataset/build.py

2508 windows

Key technical terms

- CSV dataset
- datasheet
- fixed seed
- label counts
- CPU-only

What it does

- 1614 healthy windows
- 536 H0 benign-fault windows
- 358 H1 attack windows

The discriminator separates fault response from attack response

discriminator/model.py

05

A stratified Random Forest classifies anomalous windows as H0 or H1.

discriminator/model.py

Random Forest

Key technical terms

- 90 trees
- depth 6
- 65/35 split
- ROC-AUC
- confusion matrix

What it does

- Accuracy 0.987220447284345
- ROC-AUC 0.999063829787234
- Baseline accuracy 0.3993610223642173

The twin forecasts action outcomes before touching the system

twin/model.py

06

Each candidate action gets a counterfactual time-error trajectory.

twin/model.py

P4 twin

Key technical terms

- decay/floor forecast
- fidelity score
- peak error
- steady error
- discounted advice

What it does

- Ranks candidate recoveries
- Reduces trust when telemetry is messy
- Keeps safe-default as a comparator

The healing loop commits only with evidence

healing/loop.py

07

**Detect, classify, verify,
and commit with an
auditable reason.**

healing/loop.py

P5 loop

Key technical terms

- safe default
- action shortlist
- decision budget
- audit reason
- fallback

What it does

- Chooses different actions for H0 vs H1
- Checks twin fidelity
- Records decision time and reason

The benchmark compares recovery policies, not just accuracy

benchmark/run.py

08

The loop is judged against detection-only, holdover, and failover baselines.

benchmark/run.py

P6 benchmark

Key technical terms

- success rate
- wrong action
- MTTR
- peak time error
- 2 s window

What it does

- Governed success 0.95
- Wrong-action rate 0.05
- Peak error 86.32070997977753 ns

Loop walkthrough: first detect the sync anomaly

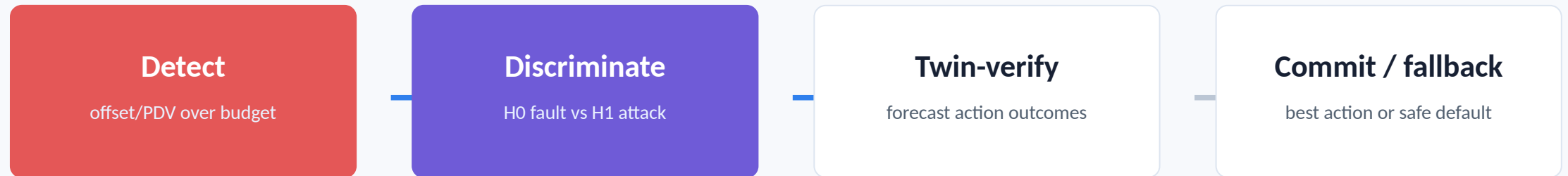
Each advance adds another stage of evidence.



The controller does nothing until timing telemetry crosses an anomaly threshold.

Loop walkthrough: then separate H0 from H1

Each advance adds another stage of evidence.



The response path diverges: benign faults should not be treated like active attackers.

Loop walkthrough: verify, commit, and audit

The progressive reveal simulates the motion of the governed control loop.



The twin compares candidate actions against the conservative safe default; low fidelity triggers fallback.

Discrimination is strong after realism hardening

The classifier beats detection-only while still making real errors.



	Pred H0	Pred H1
True H0	186	2
True H1	2	123

- REALISM_NOTES.md: removed message-type giveaways.
- Attack and benign-fault magnitudes overlap.
- Evasive attacks mimic fault signatures and can occur during congestion.

The governed loop beats simple baselines

The measured loop stays inside the 100 ns timing budget and the 2 s failure window.

Method	Recovery	Wrong action	MTTR	Peak error
always failover	0.6	0.0	0.618 s	176.369 ns
always holdover	0.6	0.4	1.18 s	176.369 ns
detection only no response	0.0	0.0	2.0 s	223.827 ns
governed loop	0.95	0.05	0.547 s	86.321 ns

Takeaway

Governed peak error is 86.32070997977753 ns, below the 100 ns budget.

Hardware next phase: timing lab first, radio later

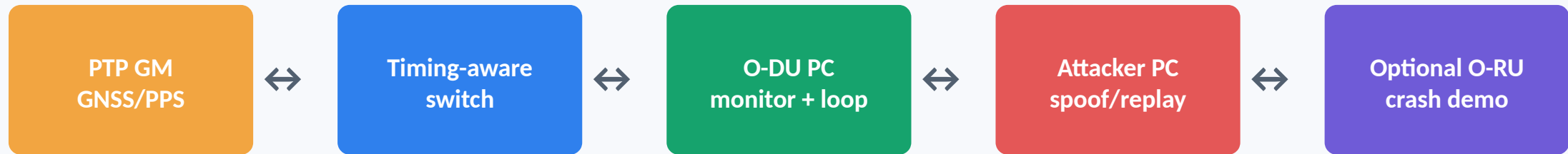
Indicative costs only; verify with vendors before procurement.

Tier A timing lab item	Purpose	Indicative cost
2-3 PCs + PTP NICs	O-DU slave, attacker, load generator	NIC \$30-\$300 each; reuse PCs
PTP/SyncE-aware switch	Boundary/transparent clock, G.8275.1	\$500 budget; \$1.5k-\$8k telecom
PTP Grandmaster or PC GM + GPSDO	Trusted reference clock	\$150-\$400 DIY; \$2k-\$10k appliance
GNSS receiver, antenna, PPS	GNSS-loss and holdover faults	\$50-\$300
SFPs, cabling, TAP/mirror	Passive PTP capture	\$100-\$400
Time-error tester	ns-true validation reference	borrow/rent; buy \$5k+

Tier A total: DIY/budget ~\$800-\$3,000 reusing PCs; professional ~\$6,000-\$18,000.

Hardware path closes the loop

Same software, real telemetry: linuxptp replaces the simulator input.



Done now

- run_all.py reproduction
- dataset + benchmark
- tests pass 3/3

Next commands

- ptp4l / phc2sys / ts2phc
- tc netem and packet capture
- BMCA allow-list actions

Do not miss

- G.8275.1 profile
- OCXO vs TCXO holdover
- GNSS sky view, PPS skew, UPS
- Tier A needs no radio licence

Close: the project is a reproducible response loop today; hardware timestamping is the next validation step.